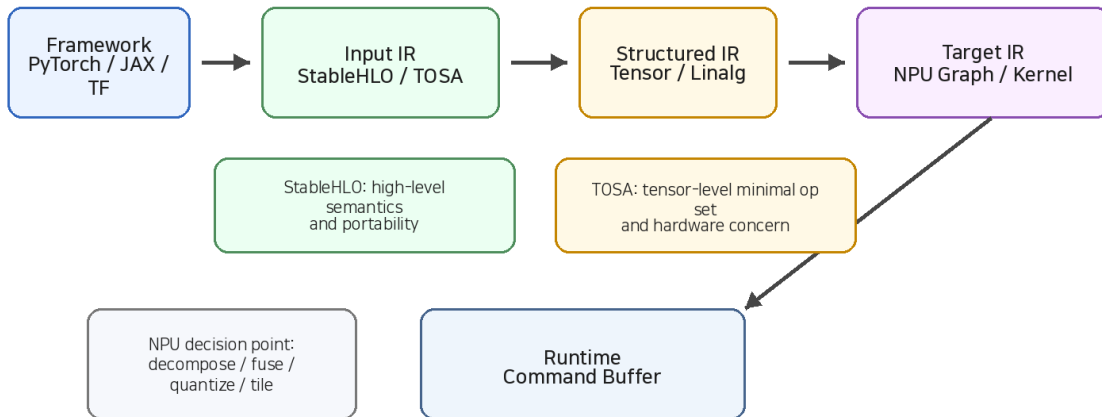


MLIR NPU Compiler - Lecture 06

StableHLO, TOSA, and AI Model Input IR

핵심 실습: LayerNorm/GELU primitive decomposition 및 NPU legal op audit

Lecture 06: StableHLO / TOSA as AI Model Input IR



1. 강의 목표

- StableHLO 와 TOSA 가 AI/NPU compiler pipeline 에서 어떤 입력 IR 역할을 하는지 설명한다.
- StableHLO/TOSA 의 abstraction level, op semantics, legalization boundary 를 비교한다.
- LayerNorm 과 GELU 를 primitive op graph 로 분해하고 NPU legal op set 기준으로 audit 한다.
- MatMul + Bias + GELU, LayerNorm 같은 transformer subgraph 의 fusion/decomposition policy 를 설계한다.

2. 6 강의 위치: framework semantics 에서 target legality 로

1-5 강에서 MLIR IR 구조, tooling, dialect 시스템을 학습했다면 6 강부터는 실제 AI 모델이 compiler 로 들어오는 경계를 다룹니다. NPU compiler 에서 입력 IR 은 단순한 파일 포맷이 아니라 semantic contract 입니다. 이 contract 를 기준으로 어떤 op 를 보존할지, 어떤 op 를 primitive 로 분해할지, 어떤 op 를 custom NPU dialect 로 legalize 할지 결정합니다.

Pipeline stage	대표 IR	NPU compiler 질문
Model export	StableHLO / TOSA	Framework semantics 가 어떤 op 와 shape 로 들어오는가?
Canonicalization	StableHLO/TOSA normalized form	동일 의미의 여러 표현을 하나로 정규화할 수 있는가?

Decomposition	primitive tensor ops	LayerNorm/GELU/Softmax 를 target legal op 로 쪼갤 수 있는가?
Structured lowering	Tensor / Linalg	loop/tile/vectorization 이 가능한 구조인가?
Target lowering	npu_graph / npu_kernel	NPU kernel, DMA, SRAM, accumulator contract 와 맞는가?

3. StableHLO 개념

StableHLO 는 high-level operation set 으로, ML framework 와 ML compiler 사이의 portability layer 로 볼 수 있습니다. NPU compiler 입장에서는 framework 별 graph 를 직접 다루는 대신 StableHLO semantics 를 기준으로 target legality 와 decomposition 을 시작할 수 있다는 장점이 있습니다.

- 대표 op: dot_general, convolution, reduce, broadcast_in_dim, reshape, transpose, elementwise arithmetic, compare, select 등.
- 강점: JAX/PyTorch/TF 등에서 넘어온 high-level semantics 를 비교적 풍부하게 보존한다.
- 주의점: high-level semantics 가 너무 오래 보존되면 NPU target legality 판단과 layout propagation 이 늦어질 수 있다.
- NPU 관점: dot_general 은 matmul/systolic mapping 후보이고, reduce/broadcast 는 LayerNorm/Softmax 같은 composite pattern 의 핵심 신호다.

4. TOSA 개념

TOSA 는 tensor-level operator set 으로, 여러 framework 의 high-level operation 을 공통 tensor operation 으로 표현하려는 목적을 갖습니다. MLIR TOSA dialect 는 TOSA specification 을 구현하며, CPU/GPU/NPU 같은 hardware target concern 을 고려한 minimal operator set 철학을 갖습니다.

- 대표 op: tosa.matmul, tosa.conv2d, tosa.reduce_sum, tosa.rsqrt, tosa.add/sub/mul, tosa.reshape/tile/transpose, tosa.rescale, tosa.table 등.
- 강점: NPU backend 에서 legal op set, quantized path, tensor-level decomposition 을 점검하기 좋다.
- 주의점: composite intent 를 너무 일찍 잃으면 fusion pattern matching 이 어려워질 수 있다.
- NPU 관점: tosa.rescale/table 은 quantized activation/requant path 와 연결되고, reduce_sum/rsqrt 는 LayerNorm lowering 의 핵심 primitive 다.

5. StableHLO vs TOSA 비교

관점	StableHLO	TOSA	NPU compiler implication
주요 역할	Framework-compiler portability layer	Tensor-level operator set	어느 단계에서 semantic intent 를 보존할지 결정
추상화 수준	High-level HLO semantics	Tensor-level primitive/minimal semantics	StableHLO 는 pattern 보존, TOSA 는 target audit 에 유리
대표 연산	dot_general, convolution, reduce, broadcast_in_dim	matmul, conv2d, reduce_sum, rsqrt, rescale, table	target legal op list 와 매핑 필요
Decomposition	Pass 로 primitive graph 로 전개	이미 primitive 에 가까운 경우 많음	fusion opportunity 보존 여부 확인
Quantization	semantic 보존 중심	rescale/table 등 quant path 에 유리	INT8/INT4 NPU path 에서는 TOSA-style audit 유용

동적 shape	dynamic dims 를 보존할 수 있음	shape legality 가 중요	edge NPU 는 static specialization 또는 guard 필요
----------	-------------------------	---------------------	--

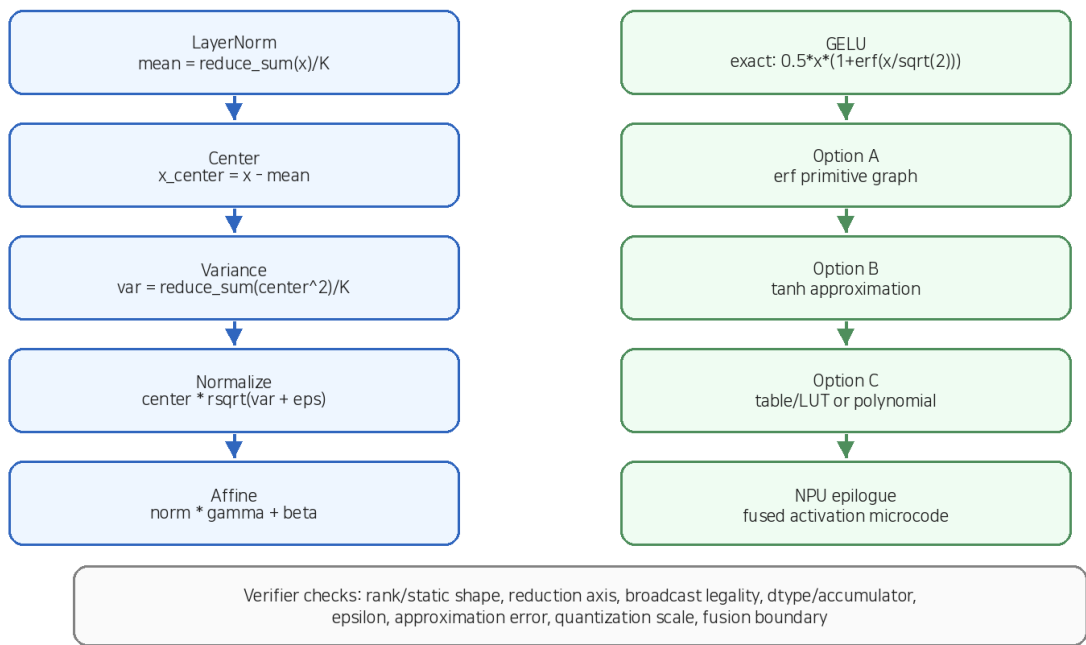
6. Composite op vs primitive op

AI 모델에는 GELU, LayerNorm, Softmax, RMSNorm 처럼 수학적으로는 하나의 layer 이지만 IR 에서는 여러 primitive op 로 나타나는 경우가 많습니다. Compiler 는 이를 무조건 분해하거나 무조건 보존하지 않습니다. 보존하면 fusion 과 numerical intent 가 유지되고, 분해하면 target legal op 와 scheduling 이 쉬워집니다.

Composite	Primitive signals	NPU lowering policy
LayerNorm	reduce_sum, sub, mul, reduce_sum, add, rsqrt, mul, add	Reduction fusion, higher-precision accumulator, gamma/beta broadcast
GELU	erf/tanh/sigmoid/table + elementwise arithmetic	MatMul epilogue fusion, LUT/polynomial approximation
Softmax	reduce_max, sub, exp, reduce_sum, reciprocal/divide, mul	Numerically stable reduction, row-wise tile scheduling
RMSNorm	mul, reduce_sum, rsqrt, mul	LayerNorm 보다 mean subtraction 없음, reduction path 최적화

7. LayerNorm decomposition

Primitive Decomposition for NPU Legalization



LayerNorm 은 transformer block 에서 매우 자주 등장합니다. NPU compiler 에서는 reduction axis, accumulator precision, epsilon 위치, gamma/beta broadcast, SRAM reuse 가 중요합니다.

LayerNorm 수식

```
# x: [B, S, K], gamma: [K], beta: [K]
mean = reduce_sum(x, axis=K) / K
center = x - broadcast(mean)
var = reduce_sum(center * center, axis=K) / K
inv_std = rsqrt(var + epsilon)
norm = center * broadcast(inv_std)
out = norm * gamma + beta
```

- Mean 과 variance reduction 은 가능하면 higher precision accumulator 를 사용한다.
- K dimension 이 vector lane 또는 NPU reduction tile 에 맞지 않으면 padding/tail policy 가 필요하다.
- Epsilon 은 variance 에 더한 뒤 rsqrt 를 적용한다.
- Broadcast 는 logical semantics 이지만, lowering 이후에는 실제 memory layout 과 vectorization 에 영향을 준다.

8. GELU decomposition

GELU 는 transformer MLP 에서 MatMul epilogue activation 으로 자주 등장합니다. NPU 가 erf 를 직접 지원하지 않는 경우 tanh approximation, sigmoid approximation, polynomial approximation, LUT/table 방식이 사용될 수 있습니다.

GELU 수식

```
# exact
GELU(x) = 0.5 * x * (1 + erf(x / sqrt(2)))

# common tanh approximation
GELU(x) ~ 0.5 * x * (1 + tanh(sqrt(2/pi) * (x + 0.044715*x^3)))
```

Option	IR pattern	NPU 장점	주의점
Exact erf	erf + add/mul	정확도 보존	special function unit 또는 느린 fallback 필요
Tanh approximation	tanh + polynomial	많은 backend 에서 구현 쉬움	오차 허용 범위 필요
LUT/table	table lookup	INT8/INT4 activation 에 유리	calibration range 와 saturation 필요
Fused epilogue	matmul + bias + gelu	memory bandwidth 절감	post-op ordering 과 accumulator dtype 확인

9. NPU legal op audit matrix

NPU Legal Op Audit Matrix

Op	StableHLO	TOSA	NPU support	Action	Test
MatMul	dot_general	matmul	MXU/Systolic	legalize to npu.matmul	shape + K alignment
LayerNorm	reduce/add/rsqrt	reduce_sum/rsqrt	partial	decompose + fuse	epsilon + axis
GELU	erf/tanh graph	tanh/table graph	approx/LUT	select approximation	max error
Softmax	reduce_max/exp/sum	exp/reduce_sum	partial	tile + stable math	overflow test
Conv2D	convolution	conv2d	direct/winograd	layout rewrite	NHWC/HWCF

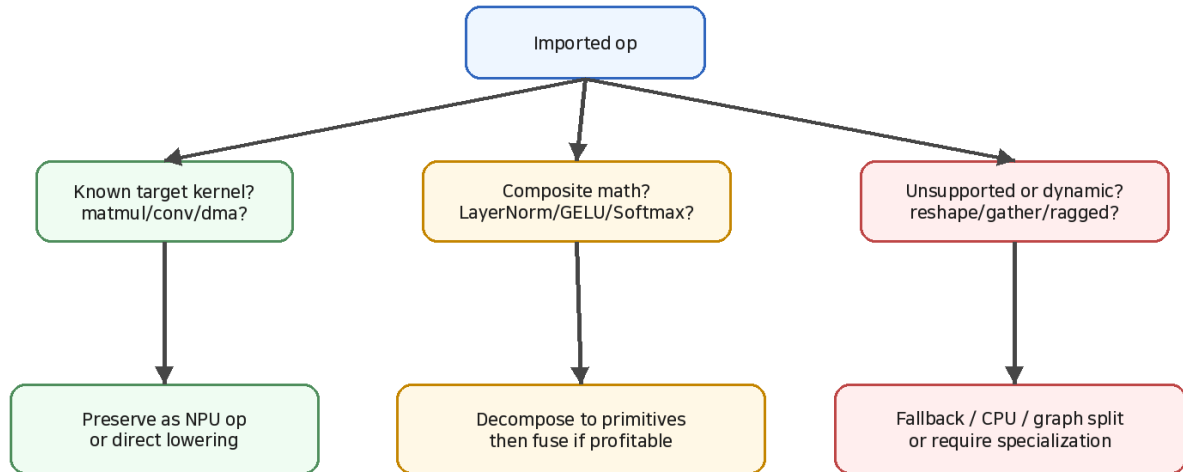
Use this matrix before writing a lowering pass: unsupported op -> decompose; supported composite -> maybe preserve and fuse.

StableHLO/TOSA importer 다음 단계에서 반드시 audit matrix 를 작성해야 합니다. 이것이 없으면 unsupported op fallback, dynamic shape, dtype mismatch, broadcast materialization 비용이 뒤늦게 발견됩니다.

Audit 항목	확인 질문	예시 결정
Op support	target kernel 이 있는가?	dot_general -> npu.matmul, reduce_sum -> npu.reduce
Shape	rank/dim 이 static 인가?	B/S dynamic 이면 specialization 또는 guard
DType	f32/f16/bf16/int8/int4?	accumulator i32/f32, output requant
Layout	logical layout 과 physical layout 차이는?	NHWC -> blocked 또는 K-major
Approximation	erf/tanh/table 오차 허용?	GELU LUT max error threshold
Fusion	SRAM/latency 이득이 있는가?	MatMul + Bias + GELU epilogue

10. Legalization pipeline 예시

Input IR Decision Tree for NPU Compiler



Rule of thumb: StableHLO/TOSA input IR is a semantic contract, not the final hardware schedule.

6 강에서 설계할 pipeline

```

stablehlo/tosa input
-> canonicalize
-> shape/legal op audit
-> decompose LayerNorm/GELU/Softmax when needed
-> match fusion candidates
-> lower supported primitives to npu_graph
-> lower npu_graph to npu_kernel/tensor/linalg path
-> bufferization/layout/vectorization/scheduling
  
```

중요한 원칙은 StableHLO/TOSA 에서 바로 command buffer 로 가지 않는 것입니다. 먼저 semantic-level legality 를 통과시키고, target-level contract 를 가진 NPU dialect 또는 structured IR 로 내려가야 합니다.

11. 실습 A: StableHLO/TOSA op classification

아래 표를 채우면서 importer 가 받은 op 를 NPU backend 관점에서 분류합니다.

Op or pattern	StableHLO/TOSA source	Supported?	Decompose?	Fusion candidate?	Test case
MatMul	dot_general / tosa.matmul				
LayerNorm	reduce + broadcast + rsqrt				
GELU	erf/tanh/table graph				
Softmax	reduce_max + exp + reduce_sum				
Dynamic reshape	dynamic_reshape / reshape				

12. 실습 B: LayerNorm primitive graph 작성

1. Input shape 를 $x[B,S,K]$, $\gamma[K]$, $\beta[K]$ 로 정의한다.
2. mean, center, variance, inv_std, norm, affine output 의 SSA value 를 순서대로 작성한다.
3. 각 단계마다 tensor shape 를 적는다.
4. NPU target 에서 reduction kernel, elementwise kernel, fused kernel 중 어디에 매핑할지 적는다.

13. 실습 C: GELU approximation policy

아래 3 가지 조건을 기준으로 GELU lowering 방식을 선택합니다.

조건	선택지	기준
정확도 우선	erf exact	모델 accuracy 민감, special function 지원
성능 우선	tanh approximation	activation throughput 우선, acceptable error
INT8/INT4	LUT/table	calibration range 와 scale 이 안정적
MatMul epilogue	fused activation	SRAM write/read 감소가 큰 경우

14. 흔한 실수

- Composite op 를 너무 일찍 쪼개서 fusion intent 를 잃는다.
- StableHLO/TOSA 의 logical broadcast 를 physical layout materialization 으로 착각한다.
- LayerNorm variance 계산에서 epsilon 위치를 잘못 둔다.
- GELU approximation 을 바꾸면서 numerical golden test 를 만들지 않는다.
- Dynamic shape 를 무시하고 static tile scheduling 을 만든다.
- TOSA rescale/table 같은 quantized path 신호를 버린다.

15. 과제

- MatMul + Bias + GELU subgraph 를 StableHLO 또는 TOSA pseudo-MLIR 로 작성한다.
- LayerNorm decomposed graph 의 shape table 을 완성한다.
- NPU legal op audit matrix 를 작성하고 unsupported op 3 개에 대한 fallback 또는 decomposition plan 을 제안한다.
- GELU exact/tanh/LUT 중 하나를 선택하고, 선택 이유와 regression test 항목을 작성한다.

16. References

StableHLO Specification: <https://openxla.org/stablehlo/spec> - StableHLO operation set, semantics, and portability layer.

StableHLO Overview: <https://openxla.org/stablehlo> - StableHLO ecosystem and framework/compiler interoperability overview.

MLIR TOSA Dialect: <https://mlir.llvm.org/docs/Dialects/TOSA/> - TOSA dialect rationale, operator set, and operation descriptions.

TOSA Specification: https://www.mlplatform.org/tosa/tosa_spec.html - Published TOSA specification versions.

MLIR Passes: <https://mlir.llvm.org/docs/Passes/> - TOSA-to-Linalg/Tensor/SCF lowering pass documentation.